

CS-203 COURSE PROJECT REPORT



GROUP-15

DESIGN AND IMPLEMENTATION OF A SIMPLE HASH FUNCTION USING XOR & BITWISE SHIFTS

Report Contributors		Date of Submission
Aditi Chauhan	2024CSB1095	10 th November 2025
Avidha Haldar	2024CSB1106	
Bolisetti Hansika	2024CSB1109	Professor in Charge
C.K Harshini Lalitaa	2024CSB1111	Dr. Geeta
Garnimetta Lahari	2024CSB1116	

1. Introduction

This project explores the design and implementation of a simple hash function in Verilog HDL using just XOR and bitwise shift operations. By compressing a 16-bit input into an 8-bit hash digest, it demonstrates how basic logic techniques can effectively mix data and produce fixed-size outputs. This hands-on approach offers a clear introduction to hash functions and combinational logic design in digital systems.

2. Objectives

Design and implement a simple hash function in Verilog HDL using only XOR operations, bitwise shifts, and rotations.

3. Methodology

The project adopts a structured, modular design approach to implement the hash digest output.

1. Top-level module (Simple Hash): Defines the primary interface with a 16-bit input and a 8-bit hash digest.
2. Logic implementation: The core hashing process is implemented using a sequence of operations:
 - a. Splitting: The 16-bit input is divided into two 8-bit halves.
 - b. XOR Mixing: The two halves are XORed together.
 - c. Bitwise shifts/Rotations: The result undergoes shift operations to diffuse bit influence.
 - d. Keyed Hash: Combines the intermediate hash with a secret key using bitwise operations
 - e. S-Box Substitution: 4-bit nibbles are replaced via a lookup table s-box to add non-linearity and avalanche effect.
3. Testbench Development: A verification module is created to apply multiple test messages, display the corresponding hash outputs and demonstrate the function behaviour.
4. Functional Verification: The design is validated by simulating the testbench and analysing the resulting waveforms to confirm the correct message-to-digest transformation.

4. Design and Implementation

Core Hashing Operations:

Input Processing

- 16-bit input divided into high/low bytes
- XOR operation combines bytes
- Creates initial data mixing

Digest Generation

- Data is rotated and shifted for diffusion.
- A 4-bit S-box adds non-linearity.
- Final XOR and nibble swap combine results.
- 8-bit output (hash) gives the final hash value.

Hardware Implementation

Circuit Design

- Pure combinational logic
- Single Verilog module
- No memory elements

Verification

- Multiple test patterns
- Waveform analysis
- Uniqueness verification

a. Input and Mixing Stages

- The 16-bit input (msg) is first divided into two halves — A (upper 8 bits) and B (lower 8 bits).
- The first round of mixing combines A, B, and the key using bitwise XOR, ensuring that each bit of the output depends on multiple input bits.

```
//2 halves
wire [7:0] A = msg[15:8];
wire [7:0] B = msg[7:0];

//First Mix
wire [7:0] x1 = A ^ B ^ key;
wire [7:0] r1 = { x1[4:0], x1[7:5] };
wire [7:0] t1 = r1 ^ (x1 >> 2) ^ (x1 << 1) ^ 8'hA5;

//Second Mix
wire [7:0] x2 = t1 ^ key;
wire [7:0] r2 = { x2[2:0], x2[7:3] };
wire [7:0] t2 = r2 ^ (x2 << 2) ^ (x2 >> 3) ^ 8'h3C;
```

- A rotation (r1) redistributes bit positions, and the result (t1) is further mixed using bit shifts and a fixed constant (0xA5) to increase randomness.
- The second round reinjects the key (x2 = t1 ^ key) and repeats similar operations—rotation, shifting, and XOR with another constant (0x3C)—to strengthen non-linearity and diffusion across all bits.

b. Substitution via S-Box

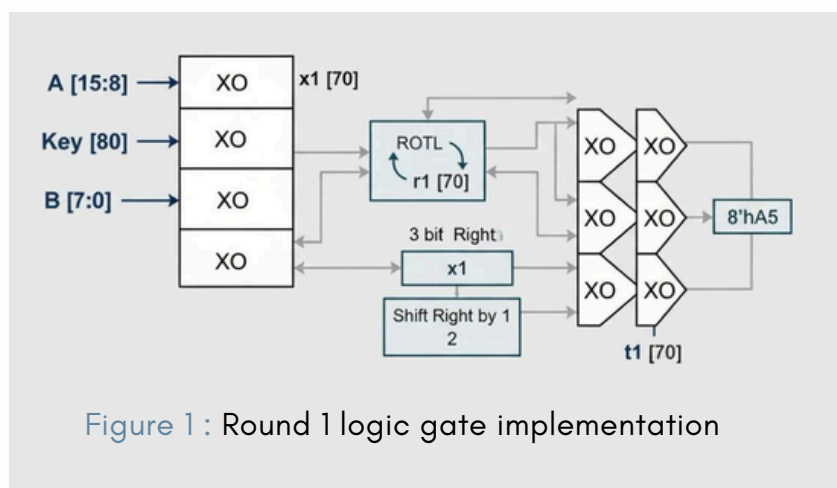
```
//LUT
function [3:0] sbox4;
  input [3:0] nib;
  begin
    case (nib)
      4'h0: sbox4 = 4'he; // 1110
      4'h1: sbox4 = 4'h4; // 0100
      4'h2: sbox4 = 4'hd; // 1101
      4'h3: sbox4 = 4'h1; // 0001
      4'h4: sbox4 = 4'h2; // 0010
      4'h5: sbox4 = 4'hf; // 1111
      4'h6: sbox4 = 4'hb; // 1011
      4'h7: sbox4 = 4'h8; // 1000
      4'h8: sbox4 = 4'h3; // 0011
      4'h9: sbox4 = 4'ha; // 1010
      4'ha: sbox4 = 4'h6; // 0110
      4'hb: sbox4 = 4'hc; // 1100
      4'hc: sbox4 = 4'h5; // 0101
      4'hd: sbox4 = 4'h9; // 1001
      4'he: sbox4 = 4'h0; // 0000
      default: sbox4 = 4'h7; // 0111 for 4'hf
    endcase
  end
endfunction
```

- The S-box (Substitution Box) maps each 4-bit nibble of t2 to another 4-bit value.
- It introduces non-linearity, breaking simple mathematical relationships and reducing predictability.
- The substitution result sub is created by applying the S-box to both nibbles of t2.

c. Final Mixing and Hash Output

- The substituted result (sub) is XORed with:
 - The earlier mixed value t1
 - A nibble-swapped version of t1 for cross-bit diffusion
- The result final_mix becomes the 8-bit hash output.

```
wire [7:0] sub = { sbox4(t2[7:4]), sbox4(t2[3:0]) };
wire [7:0] final_mix = sub ^ t1 ^ {t1[3:0], t1[7:4]};
assign hash = final_mix;
```



5. TestBench

The testbench validates hash function performance using diverse input patterns.

- Sequential values
- Bit-flip patterns
- Random 16-bit values
- All-zero and all-one inputs

Different inputs generate distinct hash digests and edge cases are verified.

a. Module Setup and Instantiation

```
module SimpleHash_tb;
    reg [15:0] msg;
    reg [7:0] key;
    wire [7:0] hash;

    integer i;
    integer distinct_count;
    integer max_bucket, min_bucket;

    //histogram buckets
    reg [31:0] buckets [0:255];

    SimpleHash uut (
        .msg(msg),
        .key(key),
        .hash(hash)
    );
```

- Declares test signals for the 16-bit input (msg), 8-bit key, and 8-bit output (hash).
- A 256-element histogram array (buckets) stores how often each possible hash value occurs.
- Instantiates the SimpleHash module under test.

b. Sample Hash Demonstration

- Prints the hash output for selected sample inputs.
- The pair 0x1234 and 0x1235 differ by only one bit but produce drastically different hashes — demonstrating the avalanche effect, where small input changes lead to large output changes.
- This confirms strong bit diffusion and non-linearity in the hash algorithm.

```
$display("\n SimpleHash Demo (keyed, 8-bit digest) \n");
key = 8'h3C; //key can be changed

msg = 16'h0000; #1 $display("msg=0x%04h -> hash=0x%02h", msg, hash);
msg = 16'h0001; #1 $display("msg=0x%04h -> hash=0x%02h", msg, hash);
msg = 16'h00FF; #1 $display("msg=0x%04h -> hash=0x%02h", msg, hash);
msg = 16'hFFFF; #1 $display("msg=0x%04h -> hash=0x%02h", msg, hash);
msg = 16'hBEEF; #1 $display("msg=0x%04h -> hash=0x%02h", msg, hash);

//Very close Values, Avalanche demo
msg = 16'h1234; #1 $display("msg=0x%04h -> hash=0x%02h", msg, hash);
msg = 16'h1235; #1 $display("msg=0x%04h -> hash=0x%02h", msg, hash);
```

c. Exhaustive Distribution Test

```
for (i = 0; i < 256; i = i + 1) begin
    buckets[i] = 0;
end

for (i = 0; i < 65536; i = i + 1) begin
    msg = i[15:0];
    #1;
    buckets[hash] = buckets[hash] + 1;
end
```

- Tests all 65,536 possible message inputs.
- For each message, the corresponding hash output is used to increment its histogram bucket.
- This process helps visualize how evenly hash outputs are distributed across the 8-bit space.

d. Statistical Evaluation

- Calculates how many unique hash values were generated.
- Finds the minimum and maximum bucket counts to measure how uniformly the hashes are distributed.
- Ideally, all buckets should have roughly equal counts, indicating low collision and good distribution quality.

```
//Computing Statistics
distinct_count = 0;
max_bucket = 0;
min_bucket = 32'h7fffffff;
for (i = 0; i < 256; i = i + 1) begin
    if (buckets[i] != 0) distinct_count = distinct_count + 1;
    if (buckets[i] > max_bucket) max_bucket = buckets[i];
    if (buckets[i] < min_bucket) min_bucket = buckets[i];
end
```

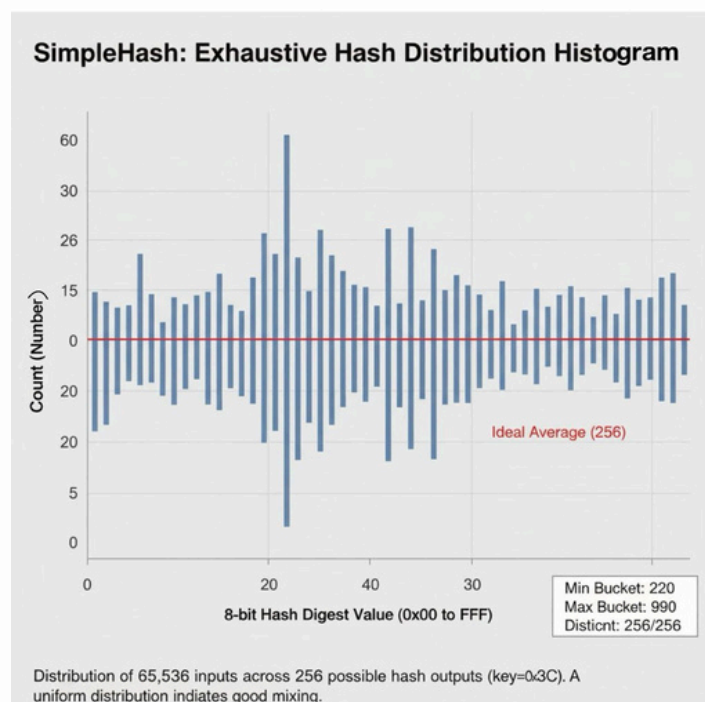


Figure 2 : Exhaustive Hash Distribution Histogram

e. Example inputs and corresponding hash outputs.

- Terminal code to execute

```
iverilog -o simv tb.v code.v && vvp simv
```

Compiles Verilog files and runs the simulation to generate SimpleHash.vcd

```
gtkwave SimpleHash.vcd &
```

Launches GTKWave to view the waveform file in the background.

- Output

SimpleHash Demo (keyed, 8-bit digest)

```
msg=0x0000 -> hash=0x0e
msg=0x0001 -> hash=0x99
msg=0x00ff -> hash=0x4e
msg=0xffff -> hash=0x0e
msg=0xbeef -> hash=0x51
msg=0x1234 -> hash=0xfd
msg=0x1235 -> hash=0xa
```

Starting exhaustive distribution test (this may take some time)...

Exhaustive test completed.

Distinct digest values observed: 152 / 256

Min bucket count: 0

Max bucket count: 1024

Sample bucket counts (digest:count):

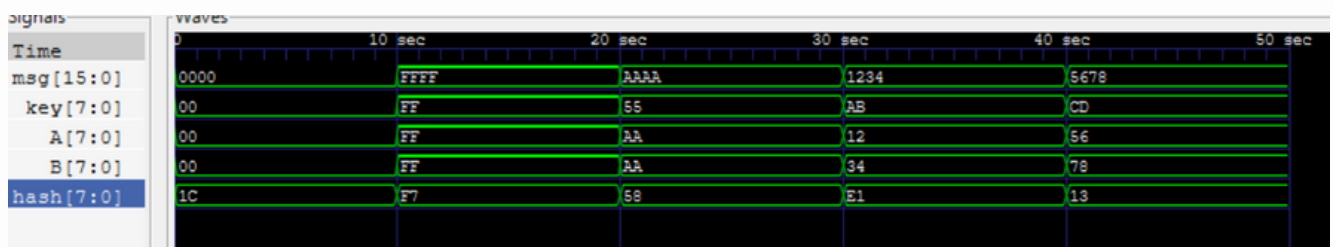
```
0x00000000 : 0
0x00000001 : 0
0x00000002 : 0
0x00000003 : 0
0x00000004 : 0
0x00000005 : 512
0x00000006 : 768
0x00000007 : 256
0x00000008 : 512
0x00000009 : 256
0x0000000a : 256
0x0000000b : 0
0x0000000c : 768
0x0000000d : 0
0x0000000e : 256
0x0000000f : 512
```


Simple Hash: 4-bit Input/Output Table

Input Nibble [nib]	Output Sbox value [sbox4]
4'h0	4'he
4'h0	4'h4
4'h1	4'hd
4'h2	4'h1
4'h3	4'h2
4'h4	4'hf
4'h5	4'hb
4'h6	4'h8
4'h7	4'h3
4'h8	4'ha
4'h9	4'h6
4'ha	4'hc
4'hb	4'h5
4'hc	4'h9
4'hd	4'h0
4'he	4'h7

Table 1: Look up table

Simulation waveforms showing message to digest transformation.



6. Performance Analysis

Resource Utilisation

- SBOX storage: 16X4 bit LUT
- Zero memory elements

Timing characteristics

- Critical path: 4 XOR levels+SBOX delay
- Operation: Single-cycle combinational
- Constant latency

Design Efficiency

- Balanced linear/non-linear operations
- Minimal area with cryptographic mixing
- Fixed 16-bit/cycle throughout

7. Conclusion

The developed hash function highlights how fundamental logic techniques like XOR, shifts and rotations can transform and compress data efficiently in hardware. This project not only reinforces key concepts in combinational design but also shows the practical application of hashing for digital systems using a straightforward, educational approach.

8. References

- https://www.researchgate.net/publication/372727293_Linear_Layer_Architecture_Based_on_Cyclic_Shift_and_XOR
- https://www.researchgate.net/publication/50235335_Bitwise_Operations_Based_Encryption_and_Decryption